

```

1  import components.simplereader.SimpleReader;
2
3  /**
4   * Program to convert an XML RSS (version 2.0) feed from a given URL into the
5   * corresponding HTML output file.
6   *
7   * @author Eric Zhou
8   */
9  public final class RSSReader {
10
11      /**
12       * Private constructor so this utility class cannot be instantiated.
13       */
14      private RSSReader() {
15      }
16
17      /**
18       * Outputs the "opening" tags in the generated HTML file. These are the
19       * expected elements generated by this method:
20       *
21       * <html> <head> <title>the channel tag title as the page title</title>
22       * </head> <body>
23       * <h1>the page title inside a link to the <channel> link</h1>
24       * <p>
25       * the channel description
26       * </p>
27       * <table border="1">
28       * <tr>
29       * <th>Date</th>
30       * <th>Source</th>
31       * <th>News</th>
32       * </tr>
33       *
34       * @param channel
35       *         the channel element XMLTree
36       * @param out
37       *         the output stream
38       * @updates out.content
39       * @requires [the root of channel is a <channel> tag] and out.is_open
40       * @ensures out.content = #out.content * [the HTML "opening" tags]
41       */
42      private static void outputHeader(XMLTree channel, SimpleWriter out) {
43          assert channel != null : "Violation of: channel is not null";
44          assert out != null : "Violation of: out is not null";
45          assert channel.isTag() && channel.label().equals("channel")
46              : "" + "Violation of: the label root of channel is a <channel> tag";
47          assert out.isOpen() : "Violation of: out.is_open";
48
49          int titleIndex = getChildElement(channel, "title"); //find the child
50          int linkIndex = getChildElement(channel, "link");
51          int descriptionIndex = getChildElement(channel, "description");
52
53          String title; //check the title child
54          if (titleIndex != -1 && channel.child(titleIndex).numberOfChildren() > 0) {
55              title = channel.child(titleIndex).child(0).label();
56          } else {
57              title = "Empty Title";
58          }
59
60          String link = channel.child(linkIndex).child(0).label();
61
62          String description; //check the description
63          if (descriptionIndex != -1
64              && channel.child(descriptionIndex).numberOfChildren() > 0) {

```

```

71      description = channel.child(descriptionIndex).child(0).label();
72  } else {
73      description = "No Description";
74  }
75
76  out.println("<html>"); //print the header
77  out.println("<head><title>" + title + "</title></head>");
78  out.println("<body>");
79  out.println("<h1><a href=\"\" + link + "\">" + title + "</a></h1>");
80  out.println("<p>" + description + "</p>");
81  out.println("<table border =\"1\">");
82  out.println("<tr><th>Date</th><th>Source</th><th>News</th></tr>");
83  }
84
85  /**
86   * Outputs the "closing" tags in the generated HTML file. These are the
87   * expected elements generated by this method:
88   *
89   * </table>
90   * </body> </html>
91   *
92   * @param out
93   *         the output stream
94   * @updates out.contents
95   * @requires out.is_open
96   * @ensures out.content = #out.content * [the HTML "closing" tags]
97   */
98  private static void outputFooter(SimpleWriter out) {
99      assert out != null : "Violation of: out is not null";
100     assert out.isOpen() : "Violation of: out.is_open";
101
102     out.println("</table>");
103     out.println("</body></html>");
104 }
105
106 /**
107  * Finds the first occurrence of the given tag among the children of the
108  * given {@code XMLTree} and return its index; returns -1 if not found.
109  *
110  * @param xml
111  *        the {@code XMLTree} to search
112  * @param tag
113  *        the tag to look for
114  * @return the index of the first child of type tag of the {@code XMLTree}
115  *         or -1 if not found
116  * @requires [the label of the root of xml is a tag]
117  * @ensures <pre>
118  * getChildElement =
119  * [the index of the first child of type tag of the {@code XMLTree} or
120  * -1 if not found]
121  * </pre>
122  */
123  private static int getChildElement(XMLTree xml, String tag) {
124      assert xml != null : "Violation of: xml is not null";
125      assert tag != null : "Violation of: tag is not null";
126      assert xml.isTag() : "Violation of: the label root of xml is a tag";
127
128      for (int i = 0; i < xml.numberOfChildren(); i++) {
129          if (xml.child(i).label().equals(tag)) { //check every tag
130              return i;
131          }
132      }
133      return -1;
134  }
135

```

```

136  /**
137   * Processes one news item and outputs one table row. The row contains three
138   * elements: the publication date, the source, and the title (or
139   * description) of the item.
140   *
141   * @param item
142   *         the news item
143   * @param out
144   *         the output stream
145   * @updates out.content
146   * @requires [the label of the root of item is an <item> tag] and
147   *         out.is_open
148   * @ensures <pre>
149   * out.content = #out.content *
150   * [an HTML table row with publication date, source, and title of news item]
151   * </pre>
152   */
153 private static void processItem(XMLTree item, SimpleWriter out) {
154     assert item != null : "Violation of: item is not null";
155     assert out != null : "Violation of: out is not null";
156     assert item.isTag() && item.label().equals("item")
157         : "" + "Violation of: the label root of item is an <item> tag";
158     assert out.isOpen() : "Violation of: out.is_open";
159
160     int dateIndex = getChildElement(item, "pubDate"); //get the index
161     int titleIndex = getChildElement(item, "title");
162     int linkIndex = getChildElement(item, "link");
163     int sourceIndex = getChildElement(item, "source");
164     int descriptionIndex = getChildElement(item, "description");
165
166     String date;
167     if (dateIndex != -1 && item.child(dateIndex).numberOfChildren() > 0) {
168         date = item.child(dateIndex).child(0).label();
169     } else {
170         date = "No date available";
171     }
172
173     String title;
174     if (titleIndex != -1 && item.child(titleIndex).numberOfChildren() > 0) {
175         title = item.child(titleIndex).child(0).label();
176     } else if (descriptionIndex != -1
177         && item.child(descriptionIndex).numberOfChildren() > 0) {
178         title = item.child(descriptionIndex).child(0).label();
179     } else {
180         title = "No title available";
181     }
182
183     String link = "";
184     if (linkIndex != -1 && item.child(linkIndex).numberOfChildren() > 0) {
185         link = item.child(linkIndex).child(0).label();
186     }
187
188     out.println("<tr>");
189     out.println("<td>" + date + "</td>");
190     if (sourceIndex != -1 && item.child(sourceIndex).numberOfChildren() > 0) {
191         String sourceUrl = item.child(sourceIndex).attributeValue("url");
192         String sourceLabel = item.child(sourceIndex).child(0).label();
193         out.println("<td><a href=\"\" + sourceUrl + \"\">\" + sourceLabel + "</a></td>");
194     } else {
195         out.println("<td>No source available</td>");
196     }
197     if (!link.isEmpty()) {
198         out.println("<td><a href=\"\" + link + \"\">\" + title + "</a></td>");
199     } else {
200         out.println("<td>" + title + "</td>");

```

```

201    }
202    out.println("</tr>");
203 }
204
205 /**
206  * Main method.
207  *
208  * @param args
209  *     the command line arguments; unused here
210  */
211 public static void main(String[] args) {
212     SimpleReader in = new SimpleReader1L();
213     SimpleWriter out = new SimpleWriter1L();
214
215     out.println("Enter the URL of an RSS feed: ");
216     String url = in.nextLine();
217
218     XMLTree xml = new XMLTree1(url); //check the input
219     if (xml.label().equals("rss") && xml.attributeValue("version").equals("2.0")) {
220         XMLTree channel = xml.child(0);
221         SimpleWriter fileOut = new SimpleWriter1L("output.html");
222         outputHeader(channel, fileOut);
223
224         for (int i = 0; i < channel.numberOfChildren(); i++) {
225             if (channel.child(i).label().equals("item")) {
226                 processItem(channel.child(i), fileOut);
227             }
228         }
229
230         outputFooter(fileOut);
231         fileOut.close();
232         out.println("generate successfully");
233     } else {
234         out.println("Invalid RSS feed.");
235     }
236
237     in.close();
238     out.close();
239 }
240
241 }
242

```